

IA Aplicada com LLMs

Aula 03 — Context engineering

Onde paramos

O deck anterior terminou com uma tabela de técnicas e uma constatação incômoda: **todas ocupam espaço.**

- few-shot infla a **entrada**
- CoT infla a **saída**
- self-consistency multiplica por **N**
- generated knowledge **dobra** o número de chamadas

Todas fazem a mesma coisa por baixo: **colocam mais coisa na janela.**

A pergunta que ficou

E quando não cabe mais?

E, mais importante — porque acontece **muito antes** de estourar:

E quando ainda cabe, mas o resultado piora?

A distinção que organiza tudo

Prompt é o que você **escreve**.

Contexto é o que o modelo **recebe**.

Na Aula 01, prompt era o que você digitava.

No exercício da Aula 02, já eram duas etapas montando dois contextos.

Num agente, o contexto é montado **em tempo de execução**.

Roteiro

1. O que realmente está na janela
2. Por que "mais" piora antes de estourar
3. **Curadoria** — o que não colocar
4. **Posição** — onde cada coisa entra
5. **O orçamento de contexto**
6. **Estático × dinâmico**

O que está na janela

1. system prompt	(você escreveu)	estável
2. exemplos few-shot	(você escolheu)	estável
3. schema de saída	(você definiu)	estável
4. histórico	(acumulou)	CRESCCE
5. documentos	(recuperados)	varia
6. saída de ferramentas	(chegou do mundo)	varia
7. a mensagem do usuário		pequena

Duas observações sobre esse desenho

1. A mensagem do usuário — o que a maioria chama de "o prompt" — é normalmente a **menor** fatia.

O que enche a janela está em 4, 5 e 6.

2. Você **escreve** os três de cima.

Os três do meio são **montados pelo seu código**.

É aí que mora a engenharia.

"Mais contexto é, no pior caso, inofensivo"

Não é.

Três razões — e **nenhuma** delas é o limite da janela.

1. Lost in the middle

A informação no **meio** do contexto é sistematicamente pior aproveitada.

Acrescentar 15 documentos irrelevantes **empurra o relevante para o meio**.

Você não adicionou ruído neutro.

Você **mudou de lugar** o que importava.

2. Contexto contraditório

Dois documentos que discordam **não** fazem o modelo dizer *"há divergência entre as fontes"*.

Fazem ele **escolher um**, sem avisar.

Recuperar "por precaução" é uma forma de introduzir contradição.

3. Diluição da instrução

A sua instrução **compete** com todo o resto pela atenção do modelo.

300 tokens de system cercados de 50 mil tokens de documento têm menos força relativa do que os mesmos 300 cercados de 2 mil.

Vendo acontecer: a mesma pergunta, três contextos

```
PERGUNTA = "meu pedido chegou?"
```

```
# (a) só a pergunta  
f"{INSTRUCAO}\n\nCliente: {PERGUNTA}"
```

```
# (b) + o que ela precisa  
f"{INSTRUCAO}\n\n{PEDIDO_ATUAL}\n\nCliente: {PERGUNTA}"
```

```
# (c) + tudo o que existe sobre o cliente  
f"{INSTRUCAO}\n\n{HISTORICO_COMPLETO}\n\nCliente: {PERGUNTA}"
```

O resultado

	O que o modelo faz
(a)	não sabe. Diz que não sabe, ou inventa um status
(b)	responde certo: pedido 48219, em transporte
(c)	piora — cita o pedido errado, mistura datas

De (a) para (b), contexto **resolveu**.

De (b) para (c), mais contexto **estragou**.

E a informação certa **estava lá** em (c), inteira e correta.
Ela só estava **no meio**.

A formulação que fica

O contexto **não é um saco** onde se joga tudo o que talvez ajude.

É uma **seleção** — e toda seleção implica **excluir**.

A pergunta de curadoria

Não é "*o que pode ajudar?*" — quase tudo pode.

Se eu remover isto, o que o modelo **deixa de conseguir fazer?**

Não sabe responder? **O item sai.**

Aplicada às fontes

Fonte	O que tirar
System	instruções que repetem o schema; adjetivos
Few-shot	exemplos redundantes — cobertura > quantidade
Histórico	turnos antigos; o resumo custa menos que o bruto
Documentos	tudo além dos poucos relevantes
Saída de ferramenta	JSON gigante devolvido inteiro

Dois casos que merecem destaque

Documentos: se você precisa de 20, o problema é de **recuperação**, não de janela. (É a aula de RAG.)

Saída de ferramenta: uma consulta que devolve 200 registros vira 8 mil tokens **a cada volta do laço**.

Ferramenta bem projetada devolve **o que o modelo precisa**, não o que a API dela retorna.

Posição importa

O critério vem direto do *lost in the middle*:

instrução crítica e pergunta no **começo ou no fim**.
Nunca soterradas no meio.

O layout

instrução + exemplos + schema	← o que o modelo precisa saber
documentos recuperados	← o material de trabalho
histórico recente + A PERGUNTA	← por último

A **pergunta fica no fim** — mesmo que já apareça no início.

Em contexto longo, repeti-la ao final é a intervenção **mais barata de escrever** e uma das que mais mudam o resultado.

O orçamento de contexto

Fonte	Teto	Ao estourar
system + schema	800	não estoura — é fixo
few-shot	600	menos exemplos, mantendo cobertura
documentos	3.000	recuperar menos, ranquear melhor
histórico	1.500	resumir os antigos
pergunta	200	—
reserva para a saída	1.000	—

Duas coisas que o orçamento resolve

- 1. A reserva para a saída.** A janela é entrada + saída. Sem reservar, você monta um contexto lindo e recebe `finish_reason="length"`.
- 2. Comportamento previsível sob carga.** Sem teto, o sistema vai bem no protótipo e degrada **em silêncio** quando as conversas crescem.

E um efeito que só aparece com o sistema rodando:
o orçamento torna o comportamento reproduzível.

Estático × dinâmico

	Estático (hoje)	Dinâmico (aula de agentes)
Quando decide	ao escrever o código	em tempo de execução
Técnicas	seleção, orçamento, posição	<i>compaction</i> , resumo de trajetória, <i>tool clearing</i> , sub-agentes
Pergunta	o que montar	o que descartar , e quando

Depois de 8 passos, a maior parte da janela de um agente é **história que ele mesmo produziu**.

Síntese

- **Prompt é o que você escreve; contexto é o que o modelo recebe.**
A mensagem do usuário costuma ser a **menor** fatia
- **Mais contexto piora antes de estourar:** *lost in the middle*,
contradição e diluição
- A pergunta: "**se eu remover isto, o que o modelo deixa de fazer?**"
- Precisa de 20 documentos? O problema é de **recuperação**
- **Posição importa:** instrução e **pergunta** no começo ou no fim, nunca no meio
- **Orçamento por fonte + reserva para a saída.** Sem teto, degrada em silêncio
- **O orçamento torna o comportamento reproduzível** — sem teto, conversas de tamanhos diferentes recebem qualidades diferentes